

INFORMAÇÕES		
MÓDULO	AULA	INSTRUTOR
07	ASP.NET Core Web API – Avançado - Padrões de Projeto, Arquitetura Limpa e Alta Performance	07 - O Pragmatismo no CQRS: Retornando Valores em Commands
		Guilherme Paracatu

1. Retornando Valores em Commands

Introdução:

Na teoria o CQRS diz que um Command (Escrita) altera o estado do sistema, mas não retorna nenhum dado. Mas se eu acabei de dar um POST para criar um Veículo, como o Front-end vai saber qual foi o Id gerado no banco de dados para redirecionar o usuário para a tela de detalhes?

Essa é a pergunta que separa os teóricos dos desenvolvedores de software do mundo real. Hoje vamos aprender como resolver isso de forma profissional.

1. A Teoria: Purismo vs. Pragmatismo

De forma bem direta e simples: **Pragmatismo é focar no que funciona na prática e traz resultados reais, em vez de seguir teorias, dogmas ou regras de forma cega.**

Uma pessoa pragmática é aquela que olha para um problema e pensa: "*Qual é a forma mais eficiente e útil de resolver isso agora?*", mesmo que a solução fuja um pouco do "manual de regras".

O que isso significa na Programação?

No mundo do desenvolvimento de software, vivemos uma guerra constante entre dois perfis: o **Purista** e o **Pragmático**.

Para usar o exemplo da nossa aula de CQRS:

- **O Desenvolvedor Purista (Teórico):** Ele lê o livro de arquitetura e segue a regra como se fosse uma religião. *A atitude dele:* "O livro diz que um Command não pode retornar NADA. Se o Front-end precisa do ID do veículo que acabou de ser salvo, azar o dele. Que ele faça uma nova requisição de consulta no banco de dados. A minha arquitetura está perfeitamente pura!"
- **O Desenvolvedor Pragmático (Realista):** Ele entende a teoria, sabe por que a regra existe, mas avalia o impacto no mundo real. *A atitude dele:* "A teoria do Command vazio é ótima, mas no nosso projeto isso vai dobrar o tráfego de rede e atrasar a equipe do Front-end. Vamos ser pragmáticos: vou quebrar essa regrinha do livro e devolver o ID logo de uma vez, porque isso é melhor para o produto final."

"Ser pragmático é entender que a Arquitetura de Software existe para servir ao Projeto, e não o Projeto para servir à Arquitetura. Se uma regra teórica está prejudicando a performance ou a equipe, o desenvolvedor pragmático adapta a regra."

O Paradigma CQS (Command Query Separation)

Criado por Bertrand Meyer, o princípio diz de forma dura:

- **Perguntas (Queries):** Retornam dados, mas não alteram o sistema.
- **Ordens (Commands):** Alteram o sistema, mas retornam void (nada).

O Problema do Mundo Real

Se formos 100% puristas, o fluxo de criação de um usuário seria assim:

1. O Front-end envia o POST (Command).
2. A API retorna 200 OK (Vazio).
3. O Front-end é obrigado a fazer uma **nova requisição** GET (Query) buscando pelo Nome ou CPF para descobrir qual foi o Id gerado.

Isso gera **Overhead de Rede** (duas idas ao servidor em vez de uma) e dor de cabeça no Front-end.

A Solução Pragmática (O Padrão de Mercado)

Até mesmo Martin Fowler (um dos pais da arquitetura de software moderna) concorda que **retornar o ID gerado** ou um **Objeto de Resultado** em um Command é uma exceção válida e necessária. O mercado adota o padrão de retornar um CommandResult padronizado.



2. A Prática: Implementando o Retorno

Para não quebrar a nossa arquitetura e manter o código limpo, vamos criar uma classe padrão de resposta. Ela servirá como um "envelope" para entregar o resultado do Command.

Passo 1: Criar o Envelope de Resultado (CommandResult)

Este objeto vai padronizar todas as respostas de POST, PUT e DELETE da sua API.

Local: ITB.Domain/Core/Messages/CommandResult.cs

Código C#

```

namespace ITB.Domain.Core.Messages;

public class CommandResult
{
    public bool Sucesso { get; private set; }
    public string Mensagem { get; private set; }
    public object? Dados { get; private set; } // O pulo do gato está aqui!

    public CommandResult(bool sucesso, string mensagem, object? dados = null)
    {
        Sucesso = sucesso;
        Mensagem = mensagem;
        Dados = dados;
    }
}

```

Passo 2: Ajustar a Interface do Handler

Nós mantemos a segurança da restrição where T : ICommand (para que ninguém passe um objeto qualquer no Handler), mas agora exigimos que o método devolva a nossa classe CommandResult.

Local: ITB.Domain/Core/Messages/Interfaces/IHandler.cs

Código C#

```

using ITB.Domain.Core.Messages;

namespace ITB.Domain.Core.Messages.Interfaces;

public interface IHandler<T> where T : ICommand
{
    // A mágica acontece aqui: A operação é de Escrita, mas devolve um Envelope!
}

```

```
Task<CommandResult> Handle(T comando);
```

```
}
```

Passo 3: Garantir que o Command use a Interface

O seu comando (os dados que vêm da requisição) continua sendo uma classe simples, mas ele precisa assinar o contrato ICommand para ser aceito pelo Handler.

Local: ITB.Application/Commands/AdicionarMarcaCommand.cs

Código C#

```
using ITB.Domain.Core.Messages.Interfaces;
```

```
public class AdicionarMarcaCommand : ICommand
{
    public string Nome { get; set; } = string.Empty;
}
```

Passo 4: Retornando o Dado no Handler

Vamos salvar a Marca no banco e devolver o ID que o PostgreSQL gerou.

Local: ITB.Application/Handlers/AdicionarMarcaHandler.cs

Código C#

```
using ITB.Domain.Core.Messages;
using ITB.Domain.Core.Messages.Interfaces;

public class AdicionarMarcaHandler : IHandler<AdicionarMarcaCommand>
{
    private readonly IMarcaRepository _marcaRepository;
    private readonly IUnitOfWork _uow;

    public async Task<CommandResult> Handle(AdicionarMarcaCommand comando)
    {
        // 1. Mapeia e cria a Entidade
```

```

var novaMarca = new Marca(comando.Nome);

// 2. Adiciona no Repositório (O EF começa a rastrear, mas o Id ainda é 0)
_marcaRepository.Adicionar(novaMarca);

// 3. Salva no Banco (Aqui o PostgreSQL gera o Id e o EF preenche o objeto na
memória)
await _uow.CommitAsync();

// 4. O Retorno Pragmático! Devolvemos o Id recém-gerado dentro da propriedade
"Dados".
return new CommandResult(
    sucesso: true,
    mensagem: "Marca cadastrada com sucesso!",
    dados: new { Id = novaMarca.Id } // Objeto anônimo contendo o ID
);
}
}

```

Local: *ITB.Domain/Core/Messages/Interfaces/IMessageBus.cs*

Código C#

```

using System.Threading.Tasks;

namespace ITB.Domain.Core.Messages.Interfaces;

public interface IMessageBus
{
    Task<CommandResult> EnviarComando<T>(T comando) where T : ICommand;
}

```

Local: ITB.Infrastructure/Bus/InMemoryBus.cs

Código C#

```
using System;

using System.Threading.Tasks;

using ITB.Domain.Core.Messages; // Importação do CommandResult
using ITB.Domain.Core.Messages.Interfaces;
using Microsoft.Extensions.DependencyInjection;

namespace ITB.Infrastructure.Bus;

public sealed class InMemoryBus : IMessageBus
{
    private readonly IServiceProvider _serviceProvider;

    public InMemoryBus(IServiceProvider serviceProvider) => _serviceProvider =
        serviceProvider;

    // 1. Mudamos a assinatura para devolver Task<CommandResult>
    public async Task<CommandResult> EnviarComando<T>(T comando) where T : ICommand
    {
        // Localiza todos os especialistas registrados para o comando T
        var handlers = _serviceProvider.GetServices<IHandler<T>>();

        CommandResult? resultado = null;

        foreach (var handler in handlers)
        {
            // Executa o handler e guarda o resultado

```

```

        resultado = await handler.Handle(comando);

        // 2. Fail-Fast: Se algum handler falhar no meio do caminho, aborta e avisa
a Controller
        if (resultado != null && !resultado.Sucesso)
        {
            return resultado;
        }
    }

    // 3. Retorna o resultado final.

    // Se a lista de handlers for vazia (erro de injeção de dependência),
devolvemos um erro amigável.

    return resultado ?? new CommandResult(false, "Nenhum manipulador (handler)
encontrado para este comando.");
}
}

```

Passo 5: A Controller

A Controller recebe o envelope e repassa para o Front-end.

Local: ITB.Api/Controllers/MarcasController.cs

Código C#

```

[HttpPost]

public async Task<IActionResult> Post([FromBody] AdicionarMarcaCommand command)
{
    // O Mediator/Bus agora nos devolve o CommandResult

    var resultado = await _bus.SendCommand(command);

    if (!resultado.Sucesso)

```

```
return BadRequest(resultado); // Retorna 400 se falhou a regra de negócio
```

```
// Retorna 200 OK (ou 201 Created) com o envelope contendo o ID!
```

```
return Ok(resultado);
```

```
}
```

 **Dica:** Pessoal, prestem atenção no uso do object? Dados. Ao usar um object genérico, nós ganhamos flexibilidade. Em um cadastro simples, podemos retornar só o { Id = 5 }. Em um cenário mais complexo, poderíamos retornar um DTO inteiro recém-criado. É uma quebra do CQRS purista? Sim. É a melhor decisão para o seu projeto comercial? Com certeza!

3. Exercício de Fixação: O Redirecionamento de Veículos

O Cenário: O Front-end está construindo a tela de "Novo Veículo". Ao clicar em "Salvar", se der tudo certo, o sistema não deve apenas mostrar uma mensagem de sucesso, ele deve redirecionar o usuário diretamente para a página de "Detalhes do Veículo" (ex: /veiculos/detalhes/105).

Para isso, o Front-end precisa desesperadamente que a sua API devolva o Id do veículo e a Placa que acabaram de ser salvos no PostgreSQL.

A Missão:

1. Altere o AdicionarVeiculoHandler para retornar um CommandResult.
2. Após o CommitAsync(), preencha a propriedade dados do CommandResult com um objeto anônimo contendo o Id e a Placa do veículo recém-criado.
3. Altere a VeiculosController para retornar esse resultado.
4. Altere todos os Handlers da sua aplicação para utilizar o CommandResult.